Redis Security Standard

Four secure single-server configurations across listener and data policy

LISTENER	CACHE	DURABLE
LOCAL	Loopback only. Disposable data, allkeys-lru, persistence off.	Loopback only. Noeviction, AOF everysec, RDB and restore proof.
NETWORK	Verified TLS and exact sources. Disposable, reproducible data only.	Verified TLS and exact sources. Persistence plus restore proof.

PUBLIC SCOPE

Reusable across clients and environments. Deployment names, CA identity, DNS suffixes, addresses, secrets, and evidence remain private inputs.

Version 1.3
August 28, 2026
Redis Open Source 8.2.9 extended release
Rocky Linux 9 and CentOS Stream 9 x86-64

Contents

This component defines four reusable Redis Open Source configurations for single-server deployments. Two independent choices select the configuration:

1. `local-cache` - loopback only; every key is disposable and reproducible.
2. `network-cache` - exact remote clients over TLS; every key is disposable and reproducible.
3. `local-durable` - loopback only; Redis data must survive restart.
4. `network-durable` - exact remote clients over TLS; Redis data must survive restart.

The standard is client-neutral. Hostnames, address space, CA identities, application names, secrets, and evidence belong in each deployment's private inventory or assurance record, never in this public repository.

Platform and version policy

The supported x86-64 platforms are:

- **Rocky Linux 9** - preferred for new deployments. Redis publishes and documents its EL9 RPM repository for Rocky Linux 9. Keep the host on Rocky's current 9.x minor release. Rocky Linux 9 security support ends 2032-05-31.
- **CentOS Stream 9** - reviewed compatibility platform for existing hosts. The same exact Redis Rocky Linux 9 RPM is installed and tested on CentOS Stream 9, but Redis does not list CentOS Stream as an officially documented RPM target. CentOS Stream 9 reaches end of life 2027-05-31, so deployments need an accepted migration or replacement plan before that date.

Both platforms pin Redis Open Source `8.2.9-1.x86_64` from the official Redis Rocky Linux 9 repository, including the full OpenPGP fingerprint and exact RPM SHA-256 digest. Redis 8.2 is an Extended release supported through 2030-09-01. Remain on the latest reviewed 8.2 patch; do not move to a shorter-lived feature series merely because it has a higher version number.

For an existing CentOS Stream 9 host, an in-place Redis upgrade and an OS migration are separate changes. The compatibility profile permits the Redis security work to proceed first, provided its package, application, backup, and rollback gates pass. Use Rocky Linux 9 for a rebuild or planned OS migration.

Security baseline

All four configurations enforce the same security controls:

- Redis runs under the vendor `redis` account, hardened systemd policy, and enforcing SELinux. No Linux capabilities or loadable modules are approved. Redis's built-in `vectorset` implementation may identify itself as a module, but its commands are explicitly denied to application ACLs.
- The default Redis user is disabled. An administrative ACL user has `+@all`; every application gets a distinct ACL user, secret, key prefix, and Pub/Sub channel prefix.
- Application ACLs allow ordinary reads, writes, transactions, Lua operations, namespaced Pub/Sub, and the non-administrative `CLIENT INFO` call commonly required by connection libraries. They deny administrative, dangerous, whole-database, enumeration, and cross-prefix operations.
- Exactly one logical database is enabled. Redis database numbers are not a tenant boundary; isolation is by separate instance plus ACL identity and prefixes.
- `maxmemory` is explicit and workload-sized. The remaining host memory must cover Redis overhead, fork copy-on-write peaks, buffers, the operating system, and every co-resident service.
- `vm.overcommit_memory=1` is required. Each Redis configuration includes `disable-thp yes`, and verification checks the effective Redis process state rather than forcing a host-wide Transparent Huge Pages setting on co-resident services.
- Debug and module commands, runtime protected-configuration changes, keyspace notifications, and unbounded slow-log payload retention are disabled.

Redis is an in-memory service and a single-server deployment has an intentional availability boundary: host or process failure interrupts the service. Use a reviewed high-availability design when the recovery objective cannot tolerate that interruption; do not repurpose this standard as a cluster design.

Data profiles

Cache

The cache profile uses `allkeys-lru`, disables AOF, and disables scheduled RDB snapshots. It is acceptable only when every key is a reproducible copy of data held in an authoritative system and the application tolerates a completely empty Redis instance after restart or replacement.

Do not place authoritative records, sessions whose loss breaks a security or business requirement, durable job state, irreplaceable rate-limit evidence, locks with recovery semantics, or idempotency state in this profile unless a specific design proves that loss is safe. If cache and durable data share an instance, select the durable profile or separate them into independently sized Redis instances.

Cache acceptance includes a cold-start test from an empty data directory, successful repopulation, observed hit/miss and eviction behavior, and proof that data loss does not violate application or security requirements.

Durable

The durable profile uses `noeviction`, AOF with `appendfsync everysec`, and RDB snapshots. Memory pressure fails writes instead of silently deleting data. AOF every second limits a common crash-loss window but does not provide zero-loss durability and does not replace tested backups.

Durable acceptance includes backup and isolated restore proof, representative key and TTL checks, application authentication, a recorded recovery time, and an explicit recovery-point objective.

Application ACL contract

For an application named `example_app` with matching key and channel prefixes, the managed ACL is equivalent to:

```
user example_app reset on #<sha256-password> resetkeys ~example_app:* resetchannels
  &example_app:* -@all +@read +@write +@transaction -@admin -@dangerous -keys -scan -randomkey
  -dbsize -vadd -vcard -vdim -vemb -vgetattr -vinfo -vlinks -vrandmember -vrem -vsetattr -vsim
  +eval +eval_ro +evalsha +evalsha_ro +publish +subscribe +unsubscribe +psubscribe +punsubscribe
   +ssubscribe +sunsubscribe +spublish +ping +echo +hello +quit +client|id +client|getname
  +client|setname +client|setinfo +client|info
```

The hash is generated locally from a one-line, root-owned mode `0600` password file containing at least 32 characters. Plaintext secrets are never written to Redis configuration, ACL files, plans, logs, or documentation. The password file remains a deployment secret and should be removed from ordinary staging after it is placed in the approved secret manager.

The standard does not promise application compatibility with every Redis command. Additions to the allow-list require a reviewed least-privilege change, an explicit threat analysis for keyless or whole-database behavior, and focused tests.

Build an immutable deployment bundle

Privileged entry points reject a normal developer checkout. Build from a clean, reviewed Git revision:

```
redis/build-bundle --output /absolute/path/config-redis-bundle
```

Copy that directory to a root-owned staging location on the target. Every directory must be `root:root` and not group/other writable; every file must have one hard link, be non-symbolic, and match `SHA256SUMS`.

On the target:

```
sudo /root/config-redis-bundle/redis/install --check-bundle
sudo /root/config-redis-bundle/redis/install
```

The installer leaves `redis.service` stopped and disabled. It never initializes or exposes Redis.

Local configurations

Choose `--data-profile cache` only after accepting the cache requirements above. Otherwise select `durable`. Review without reading either secret:

```
/root/config-redis-bundle/redis/setup --plan \
  --phase initialize \
  --model local \
  --data-profile durable \
  --admin-user redis_admin \
  --admin-password-file /root/redis-admin.password \
  --application-user example_app \
  --application-password-file /root/example-app.password \
  --application-key-prefix example_app \
  --application-channel-prefix example_app \
  --maxmemory-mib 1024
```

Apply only to a new, empty data directory by adding `sudo` to the same command. Initialization does not migrate an existing instance.

The local application connection profile is:

```
scheme: redis
host: 127.0.0.1
port: 6379
database: 0
username: example_app
password: secret-manager reference
TLS: disabled because transport never leaves loopback
```

If an application accepts only a URI, use `redis://example_app:<percent-encoded-password>@127.0.0.1:6379/0`. Treat the entire URI as a secret. Use the literal loopback address, not a hostname whose resolution can drift. Never log a raw URI or copy it into source control.

Restricted-network configurations

Start with an accepted local configuration using the same data profile. The transition adds network reachability; it must not silently change the data policy, ACLs, or memory limit.

Before the transition, require all of the following:

- one stable private DNS name reserved for the Redis endpoint;
- DNS A resolution to the exact RFC 1918 bind address from the Redis host and every application host;
- a server certificate whose SAN contains that DNS name, issued by the deployment's approved private or public CA;
- a readable certificate, private key, and complete CA chain at stable absolute paths, with renewal automation and an audited in-process TLS reload hook;
- the CA trust artifact installed explicitly on every application host;
- exact application source addresses in host and cloud/network firewalls;
- no public Redis rule and no broad private CIDR when exact sources are known;
- an evidence plan that tests one allowed source and one denied source.

The setup command validates the existing local configuration, chain, key match, DNS identity, unique DNS-to-address resolution, certificate lifetime, Redis account readability, and exact private addresses. After the client-only systemd allowlist is active, target-side TLS and authenticated probes connect to `127.0.0.1` while sending and verifying the production server name. The verifier also validates the actively served chain and hostname and requires the served leaf identity to match the configured certificate. The server's private bind address is never added to `IPAddressAllow` as a self-probe workaround. The configuration sets `port 0`, `tls-port 6379`, TLS 1.2/1.3, server authentication, and ACL authentication. Client certificates are not required by this standard; mutual TLS is a separate reviewed profile.

Review the durable transition below, or select `--data-profile cache` when the accepted local source is cache-only:

```
/root/config-redis-bundle/redis/setup --plan \
  --phase expose-network \
  --data-profile durable \
  --admin-user redis_admin \
  --admin-password-file /root/redis-admin.password \
  --application-user example_app \
  --application-password-file /root/example-app.password \
  --application-key-prefix example_app \
  --application-channel-prefix example_app \
  --maxmemory-mib 1024 \
  --bind-address 10.20.30.40 \
  --server-host redis.internal.example \
  --tls-certificate-file /etc/pki/redis/server.crt \
  --tls-private-key-file /etc/pki/redis/server.key \
  --tls-ca-file /etc/pki/redis/ca-chain.crt \
  --allowed-client-address 10.20.30.21 \
  --allowed-client-address 10.20.30.22 \
  --network-controls-confirmed
```

Apply with the same options and `sudo`. The confirmation means the operator reviewed the external controls; it does not prove the command configured them. Target-side loopback verification is not allowed-path evidence. Acceptance remains pending until a real listed client records the TLS/authenticated allowed path and an independent, non-allowlisted source records the denied path.

The network application connection profile becomes:

```
scheme: rediss
host: redis.internal.example
port: 6379
database: 0
username: example_app
```

```
password: secret-manager reference
CA file: deployment-managed absolute path
certificate verification: required
TLS server name: redis.internal.example
```

The CA path and strict hostname verification must be explicit in the client library. Do not use global certificate exceptions, disabled hostname validation, IP-literal TLS names, or a system-wide insecure fallback.

Add another application identity

Applications must not share Redis credentials or prefixes. Specify the active listener model and data profile so the verifier checks the whole selected configuration:

```
sudo /root/config-redis-bundle/redis/setup \
  --phase add-application-user \
  --model local \
  --data-profile durable \
  --admin-user redis_admin \
  --admin-password-file /root/redis-admin.password \
  --application-user second_app \
  --application-password-file /root/second-app.password \
  --application-key-prefix second_app \
  --application-channel-prefix second_app
```

For the network model, select `--model network` and also provide `--server-host` and `--tls-ca-file`. The command refuses an existing identity, loads the ACL file atomically, and verifies allowed and denied operations.

Verification and evidence

Run the installed verifier with `--model local|network` and `--data-profile cache|durable`, plus the same identity and prefix values. For an exact configuration comparison, include `--maxmemory-mib`; for the network model also supply the bind, DNS, certificate/key/CA, and a root-controlled file containing one allowed client IPv4 address per line.

The verifier checks:

- exact package version, service state, ownership/modes, SELinux enforcement, systemd policy, and optional exact rendered configuration;
- disabled default user and exact password hashes/ACL rules for the selected administrative and application identities;
- successful application read/write, Lua, Pub/Sub, and `CLIENT INFO` operations within the namespace;
- denial of unauthenticated access, cross-prefix writes, `CONFIG`, and `KEYS`;
- no unapproved loadable modules, one logical database, the exact cache or durable memory/persistence policy, host overcommit, effective Redis process THP state, and listener scope;
- unique private-DNS resolution to the bind address, loopback TLS with the production SNI, the actively served certificate chain, hostname and leaf identity, and rejection of plaintext TCP for the network model.

Capture its output, `systemctl status redis`, certificate metadata without private keys, DNS results, firewall rule identifiers, and dated connection tests in the private deployment evidence set.

Zero-downtime TLS certificate adoption

Redis 8.2 can replace its in-memory OpenSSL context through the runtime `tls-cert-file` configuration without stopping its listeners or disconnecting existing clients. The installed `reload-redis-tls` helper uses the protected Unix socket and administrative ACL rather than the certificate-dependent TCP path. It re-applies the stable certificate path, verifies a new loopback TLS handshake with the production server name, and requires the Redis process ID to remain unchanged.

Review a deployment-specific invocation without reading the administrative secret:

```
/usr/local/sbin/reload-redis-tls --plan \
  --admin-user redis_admin \
  --admin-password-file /root/redis-admin.password \
  --server-host redis.internal.example \
  --tls-certificate-file /etc/pki/redis/server.crt \
  --tls-private-key-file /etc/pki/redis/server.key \
  --tls-ca-file /etc/pki/redis/ca-chain.crt
```

The renewal owner must validate and preserve the prior certificate before atomically replacing the stable file. After replacement, run the same command with `sudo`. The reload helper accepts either the standard root-owned mode `0600` password file or a root-owned mode `0400` systemd credential, allowing a renewal service to use `LoadCredential=` without copying the secret or exposing it in process arguments. If adoption or served-identity verification fails, restore the prior certificate atomically and invoke the helper again. Redis constructs the replacement TLS context before swapping it, so a rejected runtime update leaves the existing context active. A process restart is a documented recovery fallback, not the routine certificate-renewal mechanism.

Operations and change control

- For durable profiles, back up Redis persistence files from a consistent snapshot or reviewed Redis-aware procedure. Encrypt backups, restrict access, define retention, keep a copy outside the host's failure domain, and test an isolated restore.
- For cache profiles, prove rebuildability instead of treating incidental files as backups. Alert on unexpected RDB/AOF creation.
- Monitor rejected connections, memory, fragmentation, cache hit/miss and eviction rates, latency, certificate expiry, and service restarts without logging commands, keys, values, passwords, or connection URIs. Durable profiles additionally monitor AOF/RDB status and backup success.
- Rotate application passwords one identity at a time using a reviewed overlap procedure rather than hand-editing the managed ACL.
- CA renewal automation must preserve paths, ownership, modes, SELinux access, and the DNS SAN; preserve the prior leaf; atomically install the renewed chain; invoke `reload-redis-tls`; verify the served leaf and unchanged process ID; and restore plus reapply the prior leaf on failure.
- Upgrades require a new reviewed package manifest, full key and digest validation, release-note/security review, durable backup and restore proof where applicable, application staging, and a rollback window. Never convert the disabled pinned repository into an unattended update channel.

Reproduce the PDF handoff

This Markdown file is canonical. With ReportLab installed, validate and render the public handoff deterministically from the repository root:

```
redis/build-security-standard-pdf.py --check
redis/build-security-standard-pdf.py
```

The stable output is `output/pdf/redis-security-standard.pdf`. PDF acceptance also requires rendering every page to images and visually checking typography, wrapping, headers, footers, page references, and placeholder absence.

References

- Redis Open Source version management
- Redis RPM installation
- Redis security
- Redis TLS
- Redis 8.2 runtime TLS configuration
- Redis atomic TLS context replacement
- Redis ACLs
- Redis persistence
- Redis eviction
- CentOS Stream lifecycle
- Rocky Linux lifecycle

> Acceptance reminder: restricted-network deployment is incomplete until both an allowed-source test and a denied-source test are recorded in the private evidence set.